

Module 5: Java Collections Framework

This note teaches Module 5 using general Java knowledge. The course document was used only to identify the Module 5 topic list.

Learning Goals

By the end of this module, you should be able to:

- Explain what the Java Collections Framework is.
- Use `List`, `Set`, and `Map`.
- Choose the right collection for a common problem.
- Understand mutability and immutability.
- Iterate through collections.
- Recognize basic performance differences between collection types.

1. What Is The Java Collections Framework?

The Java Collections Framework is Java's built-in set of tools for storing and working with groups of objects.

Instead of manually managing arrays for every problem, you can use collection types like:

```
List<String> names = new ArrayList<>();
Set<Integer> ids = new HashSet<>();
Map<String, Integer> scores = new HashMap<>();
```

Think of it as Java's toolbox for storing multiple values.

2. List

A `List` stores items in order and allows duplicates.

```
import java.util.ArrayList;
import java.util.List;

public class ListExample {
    public static void main(String[] args) {
        List<String> students = new ArrayList<>();

        students.add("Asha");
        students.add("Ben");
        students.add("Asha");

        System.out.println(students.get(0)); // Asha
        System.out.println(students);        // [Asha, Ben, Asha]
    }
}
```

Use a `List` when:

- Order matters.

- Duplicates are allowed.
- You need to access items by position.

Common implementations:

```
ArrayList<String> names = new ArrayList<>();
LinkedList<String> queue = new LinkedList<>();
```

Most of the time, use `ArrayList` .

3. Set

A `Set` stores unique values. It does not allow duplicates.

```
import java.util.HashSet;
import java.util.Set;

public class SetExample {
    public static void main(String[] args) {
        Set<String> emails = new HashSet<>();

        emails.add("a@example.com");
        emails.add("b@example.com");
        emails.add("a@example.com");

        System.out.println(emails);
    }
}
```

The duplicate email is ignored.

Use a `Set` when:

- Values must be unique.
- You need fast existence checks.
- You do not want duplicates.

Common implementations:

```
HashSet<String> ids = new HashSet<>();
LinkedHashSet<String> ordered = new LinkedHashSet<>();
TreeSet<String> sorted = new TreeSet<>();
```

`HashSet` is fast but unordered.

`LinkedHashSet` keeps insertion order.

`TreeSet` keeps values sorted.

4. Map

A `Map` stores key-value pairs.

```
import java.util.HashMap;
import java.util.Map;

public class MapExample {
    public static void main(String[] args) {
        Map<String, Integer> inventory = new HashMap<>();

        inventory.put("laptop", 10);
        inventory.put("mouse", 25);

        System.out.println(inventory.get("laptop")); // 10
    }
}
```

Use a `Map` when:

- You want to look something up by a key.
- Each key connects to a value.
- You need fast retrieval.

Example:

```
Map<Integer, String> users = new HashMap<>();

users.put(101, "Maria");
users.put(102, "David");

System.out.println(users.get(102)); // David
```

5. Mutability And Immutability

Mutable collections can be changed after creation.

```
List<String> names = new ArrayList<>();
names.add("Sam");
names.remove("Sam");
```

Immutable collections cannot be changed after creation.

```
List<String> roles = List.of("ADMIN", "USER", "MANAGER");

// roles.add("GUEST"); // UnsupportedOperationException at runtime
```

Use immutable collections when you want safety and predictability.

6. Iteration Patterns

Enhanced For Loop

```
for (String name : students) {  
    System.out.println(name);  
}
```

forEach

```
students.forEach(name -> System.out.println(name));
```

Iterating Through A Map

```
for (Map.Entry<String, Integer> entry : inventory.entrySet()) {  
    System.out.println(entry.getKey() + ": " + entry.getValue());  
}
```

You can also loop through only keys:

```
for (String product : inventory.keySet()) {  
    System.out.println(product);  
}
```

Or only values:

```
for (Integer quantity : inventory.values()) {  
    System.out.println(quantity);  
}
```

7. Performance Basics

Here is the practical version:

Collection	Best For	Notes
ArrayList	Reading by index	Fast for access, slower for middle insert/remove
LinkedList	Frequent inserts/removes at ends	Slower for random access
HashSet	Unique values and fast lookup	No guaranteed order
LinkedHashSet	Unique values with insertion order	Slightly more overhead
TreeSet	Unique sorted values	Slower than HashSet
HashMap	Fast key-value lookup	No guaranteed key order
LinkedHashMap	Key-value lookup with insertion order	Useful for ordered reports
TreeMap	Sorted keys	Slower than HashMap

8. Choosing The Right Collection

Use this rule of thumb:

Need	Use
Ordered items with duplicates	<code>ArrayList</code>
Unique values	<code>HashSet</code>
Unique values in sorted order	<code>TreeSet</code>
Key-value lookup	<code>HashMap</code>
Key-value lookup with sorted keys	<code>TreeMap</code>
Fixed unchangeable values	<code>List.of</code> , <code>Set.of</code> , <code>Map.of</code>

Quick Check

Answer these before moving to the exercises:

1. You need to store student names in the order they registered. Which collection?
2. You need to make sure no duplicate email addresses are saved. Which collection?
3. You need to look up a product price by product ID. Which collection?
4. You need all usernames sorted alphabetically. Which collection?

Suggested answers:

1. `ArrayList`
2. `HashSet`
3. `HashMap`
4. `TreeSet`

Practice Exercises

Exercise 1: List Practice

Create an `ArrayList<String>` of student names.

Tasks:

- Add 5 names.
- Print all names.
- Remove one name.
- Check whether a specific name exists.
- Print the total number of students.

Exercise 2: Unique Emails With Set

Create a `HashSet<String>` for email addresses.

Tasks:

- Add several emails, including duplicates.
- Print the set.

- Check whether a certain email exists.
- Explain why duplicates are not stored.

Exercise 3: Product Price Lookup

Create a `HashMap<String, Double>` where the key is product name and the value is price.

Tasks:

- Add 5 products.
- Print the price of one product.
- Update the price of one product.
- Remove one product.
- Print all products and prices.

Exercise 4: Word Frequency Counter

Given a sentence:

```
String sentence = "java is fun and java is powerful";
```

Use a `Map<String, Integer>` to count how many times each word appears.

Expected output:

```
java = 2
is = 2
fun = 1
and = 1
powerful = 1
```

Exercise 5: Remove Duplicate Numbers

Given:

```
List<Integer> numbers = List.of(4, 2, 7, 2, 4, 9, 1);
```

Tasks:

- Convert it into a `Set` .
- Print unique numbers.
- Try both `HashSet` and `TreeSet` .
- Observe the difference in ordering.

Exercise 6: Student Grade Book

Create a `Map<String, List<Integer>>` .

```
Map<String, List<Integer>> grades = new HashMap<>();
```

Tasks:

- Store each student's name with a list of grades.

- Print each student's grades.
- Calculate each student's average.

Example structure:

```
Asha -> [90, 85, 92]
Ben -> [70, 75, 80]
```

Exercise 7: Inventory System

Create a small inventory program using `Map<String, Integer>` .

Tasks:

- Add items and quantities.
- Sell an item by reducing quantity.
- Restock an item by increasing quantity.
- Print low-stock items, for example quantity below `5` .

Exercise 8: Sorted Names

Use a `TreeSet<String>` .

Tasks:

- Add 8 names in random order.
- Print them.
- Notice that they come out sorted.
- Try adding a duplicate and observe what happens.

Exercise 9: Compare ArrayList And LinkedList

Create both:

```
List<Integer> arrayList = new ArrayList<>();
List<Integer> linkedList = new LinkedList<>();
```

Tasks:

- Add numbers from 1 to 10.
- Remove the first item.
- Add an item at the beginning.
- Print both lists.

This helps you understand that different `List` implementations behave similarly but may perform differently.

Exercise 10: Mini Project: Contact Book

Build a console-based contact book.

Use:

```
Map<String, String> contacts = new HashMap<>();
```

Tasks:

- Add contact name and phone number.
- Search by name.
- Update phone number.
- Delete contact.
- Print all contacts.

Example:

```
Asha -> 555-1234
Ben  -> 555-9876
```

Recommended Practice Order

1. ArrayList student names
2. HashSet unique emails
3. HashMap product prices
4. Word frequency counter
5. Student grade book
6. Contact book mini project

Lab: Library Book Tracker

Build a Java program that tracks books, borrowed books, and borrowers using collections.

Goal

Practice:

```
List
Set
Map
ArrayList
HashSet
HashMap
iteration
```

Scenario

You are building a simple library system.

The system should:

1. Store all books.
2. Prevent duplicate book IDs.
3. Track which student borrowed which book.
4. Print reports.

Starter Code

```
import java.util.*;

public class LibraryBookTracker {
```



```

public static void main(String[] args) {
    List<String> books = new ArrayList<>();
    Set<Integer> bookIds = new HashSet<>();
    Map<Integer, String> bookTitles = new HashMap<>();
    Map<String, List<String>> borrowedBooks = new HashMap<>();

    // Write your lab code here
}
}

```

Task 1: Add Books

Add 5 books to the `books` list.

```

books.add("Java Basics");
books.add("Spring Boot Guide");
books.add("SQL Fundamentals");
books.add("Git Essentials");
books.add("Clean Code");

```

Print all books.

Task 2: Add Book IDs

Add IDs to the `bookIds` set.

```

bookIds.add(101);
bookIds.add(102);
bookIds.add(103);
bookIds.add(101);

```

Print the set and notice that duplicate `101` appears only once.

Task 3: Map Book IDs To Titles

Use `bookTitles` :

```

bookTitles.put(101, "Java Basics");
bookTitles.put(102, "Spring Boot Guide");
bookTitles.put(103, "SQL Fundamentals");

```

Print the title for book ID `102` .

Task 4: Track Borrowed Books

Use this structure:

```

Map<String, List<String>> borrowedBooks = new HashMap<>();

```

Example:

```
borrowedBooks.put("Asha", new ArrayList<>());  
borrowedBooks.get("Asha").add("Java Basics");  
borrowedBooks.get("Asha").add("Clean Code");
```

Add borrowed books for at least 3 students.

Task 5: Print Borrowing Report

Expected output:

```
Asha borrowed:  
- Java Basics  
- Clean Code  
  
Ben borrowed:  
- SQL Fundamentals
```

Task 6: Find Students Who Borrowed More Than One Book

Loop through the map and print students whose borrowed book list has more than 1 item.

Task 7: Reflection Questions

Answer these:

1. Why did we use `List` for books?
2. Why did we use `Set` for book IDs?
3. Why did we use `Map<Integer, String>` for book titles?
4. Why did we use `Map<String, List<String>>` for borrowed books?

Bonus Challenge

Add a menu with `Scanner` :

1. Add book
2. Borrow book
3. Print all books
4. Print borrowed books
5. Exit

This turns the lab into a small interactive console app.

Extra Lab: Student Records Manager

Build a console-based Java program that manages students, their enrolled courses, and grades using Java collections.

Collections To Use

```
List<String> students  
Set<String> courses
```

```
Map<String, List<Integer>> studentGrades
Map<String, Set<String>> studentCourses
```

Starter Code

```
import java.util.*;

public class Module5CollectionsLab {
    public static void main(String[] args) {
        List<String> students = new ArrayList<>();
        Set<String> courses = new HashSet<>();
        Map<String, List<Integer>> studentGrades = new HashMap<>();
        Map<String, Set<String>> studentCourses = new HashMap<>();

        // Your code goes here
    }
}
```

Part 1: Add Students

Add at least 4 students.

```
students.add("Asha");
students.add("Ben");
students.add("Carlos");
students.add("Dina");
```

Print all students.

Part 2: Add Courses

Add at least 4 courses.

```
courses.add("Java");
courses.add("SQL");
courses.add("Spring Boot");
courses.add("Git");
```

Try adding "Java" twice and observe what happens.

Part 3: Assign Courses To Students

Use `Map<String, Set<String>>` .

```
studentCourses.put("Asha", new HashSet<>());
studentCourses.get("Asha").add("Java");
studentCourses.get("Asha").add("SQL");
```

Assign at least 2 courses to each student.

Part 4: Add Grades

Use `Map<String, List<Integer>>` .

```
studentGrades.put("Asha", new ArrayList<>());
studentGrades.get("Asha").add(90);
studentGrades.get("Asha").add(85);
studentGrades.get("Asha").add(92);
```

Add at least 3 grades for each student.

Part 5: Print Student Report

Print each student with:

```
Student: Asha
Courses: [Java, SQL]
Grades: [90, 85, 92]
Average: 89.0
```

You will need to loop through the grades and calculate the average.

Part 6: Find Top Student

Find the student with the highest average grade.

Expected output example:

```
Top Student: Asha with average 89.0
```

Part 7: Low Performer List

Print students whose average is below `75` .

Example:

```
Students needing support:
Ben
```

Part 8: Collection Choice Reflection

Answer:

1. Why is `List` good for grades?
2. Why is `Set` good for courses?
3. Why is `Map` good for connecting students to grades/courses?
4. What would change if you used `TreeSet` instead of `HashSet` for courses?

Module 5 Checklist

Before moving on, make sure you can:

- Create and use an `ArrayList` .
- Create and use a `HashSet` .
- Create and use a `HashMap` .
- Explain when duplicates are allowed.
- Explain when order matters.
- Loop through a `List` , `Set` , and `Map` .
- Choose the right collection for a problem.
- Build a small console app using multiple collections together.